

Introspection of Closure Types

Document #: P3273R0
Date: 2024-05-20
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Andrei Alexandrescu
<andrei@nvidia.com>
David Olsen
<dolsen@nvidia.com>
Daveed Vandevoorde
<daveed@edg.com>
Barry Revzin
<barry.revzin@gmail.com>
Michael Garland
<mgarland@nvidia.com>

Contents

[1 Introduction](#)

[2 Motivation](#)

[3 Proposed Changes: Specify Storage of Reference Captures](#)

[4 Discussion](#)

[5 References](#)

§ 1 Introduction

Recent proposals of reflection facilities for C++ (such as [\[P2996R2\]](#) and [\[P3157R0\]](#)) raise important questions regarding the applicability of reflection to the layout of closure types. The ability to introspect closure types has important applications to applications that need to carry computation and data—packaged together in closures—across address spaces: inter-process communication, networking, serialization, and GPU execution. We propose to strengthen the layout guarantees of closure types in ways that allow introspection to work appropriately. To the best of our knowledge, the guarantees we propose are already observed by current implementations, so we estimate no or low impact on existing compiler infrastructure.

§ 2 Motivation

Closure objects are a simple, syntactically compact, and convenient means to package computation and data together. Captures allow closures to carry arbitrary amounts of data within. Because of these

advantages, closures are used extensively in C++ code either as a means to customize algorithms, or in various applications of the [Command](#) design pattern.

The CUDA C++ dialect aimed at running algorithms on GPU devices with dedicated computational and memory hardware pays special attention to closure types, making them transparently available for execution on GPU hardware. Libraries such as [Thrust](#) and [Cub](#) make good use of device lambda functions as a central feature.

Introspection of closure types promises new opportunities for such codes as well as any code that needs to marshal data and computation across memory address spaces. To realize such opportunities, closure types must make their state available for introspection so that custom code can carry marshaling operations, or disallow certain data statically or dynamically.

§ 3 Proposed Changes: Specify Storage of Reference Captures

Given that the introspection primitives proposed in P2996 can query class types and that closure types are class types (per 7.5.5.2 [\[expr.prim.lambda.closure\]](#)), it follows that introspection should apply by definition to closure types. However, there is one specific issue with reference captures: per 7.5.5.3 [\[expr.prim.lambda.capture\]](#), the language definition leaves it to the implementation whether member variables are declared in the closure type to accommodate those captures. This makes it impossible for code that uses introspection for closure objects to portably take account of all captured data.

We propose to change wording in 7.5.5.3 [\[expr.prim.lambda.capture\]](#)/12:

12 An entity is *captured by reference* if it is implicitly or explicitly captured but not captured by copy. ~~It is unspecified whether additional unnamed non-static data members are declared in the closure type for entities captured by reference. If declared, such non-static data members shall be of literal type.~~ For each entity captured by reference, an unnamed non-static data member is declared in the closure type. The declaration order of these members is unspecified. The type of such a data member is an lvalue reference to:

- (12.1) — the referenced type if the entity is a reference to an object,
- (12.2) — the referenced function type if the entity is a reference to a function, or
- (12.3) — the type of the corresponding captured entity otherwise.

§ 4 Discussion

As mentioned, to our knowledge, all implementations already implement by-reference lambda captures by means of reference or pointer members, so we anticipate no or small impact on compiler implementations.

To the extent the intent of the existing wording was meant to allow optimizations (e.g., use direct access for reference captures when the lambda is executed directly upon introduction), we note that all optimizations are still possible if reflection facilities are not used for any given lambda.

§ 5 References

[P2996R2] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-02-15. Reflection for C++26.

<https://wg21.link/p2996r2>

[P3157R0] Andrei Alexandrescu, Bryce Lebach, Michael Garland. 2024-02-15. Generative Extensions for Reflection.

<https://wg21.link/p3157r0>